CS 340 Project Two – README

Clayton Cooper - CS 340 – Client/Server Development | Southern New Hampshire University

1. Project Overview

An multinational rescue-animal training company called Grazioso Salvare finds dogs that are appropriate for search and rescue operations. The purpose of this full-stack online application is to enable Grazioso Salvare employees to peruse, filter, and display shelter-animal data from five animal shelters located in the Austin, Texas, region. The Model-View-Controller (MVC) design pattern is used in this project. The Austin Animal Center Outcomes dataset, which is kept in a MongoDB database, serves as the model. The View is an interactive Dash dashboard created with JupyterDash and displayed within a Jupyter Notebook. All queries between the view and the database are mediated by the Controller, a Python CRUD module (AnimalShelter.py).

2. Required Functionality

2.1 Interactive Filter Options

Users can filter the dataset by one of four rescue types using radio buttons. Every choice initiates a MongoDB query that simultaneously updates every dashboard widget. The Water Rescue filter comes back undamaged. Female Labrador Retriever Mix, Chesapeake Bay Retriever, or Newfoundland dogs between the ages of 26 and 156 weeks. Intact male dogs of the German Shepherd, Alaskan Malamute, Old English Sheepdog, Siberian Husky, or Rottweiler breeds in the same age range are returned by the Mountain or Wilderness Rescue filter. Intact male canines of the Doberman Pinscher, German Shepherd, Golden Retriever, Bloodhound, or Rottweiler

breeds between 20 and 300 weeks of age are returned by the Disaster or Individual Tracking filter. The dashboard's unfiltered view of every animal is restored by selecting the Reset option.

2.2 Interactive Data Table

With a number of user-friendly features, the Dash DataTable component shows all returned records. The table has native multi-column sorting, per-column type-to-filter capability, and pagination at 10 rows per page. The geolocation map is driven by single-row selection, whereby clicking a record adds a marker to the animal's location. To enhance readability, alternating row shading and a blue backdrop are used for column selection highlighting.

2.3 Charts

The active filter causes two charts to dynamically update. The top ten breeds in the presently presented data are displayed in the Breed Distribution Pie Chart, a donut-style chart created with Plotly Express. It always displays the visible rows rather than the entire dataset because it is powered by the derived_virtual_data output of the data table. The Geolocation Map is an interactive Dash-Leaflet map with Austin, Texas at its center. A marker is positioned at the animal's location_lat and location_long coordinates when a record in the data table is selected. The breed of the animal is shown in a tooltip, and its name appears in a popup.

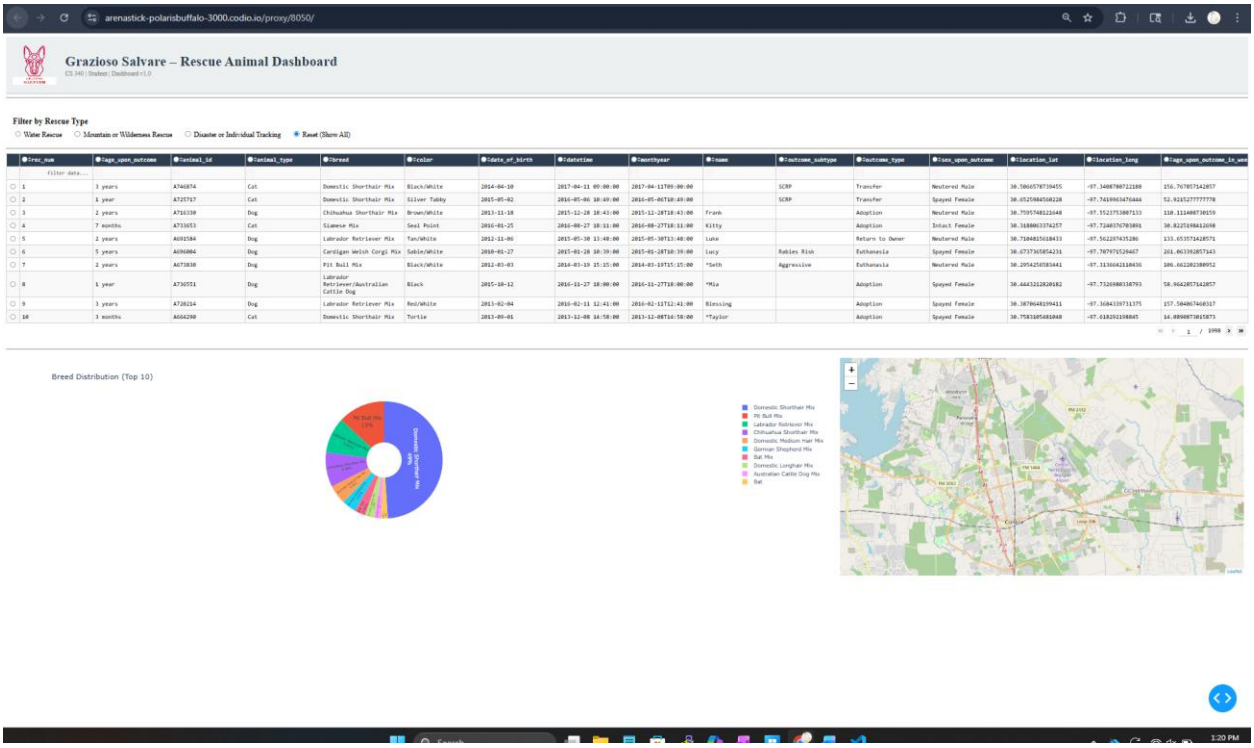
2.4 Branding

The application title and the Grazioso Salvare emblem are shown in the dashboard header. Every dashboard page has a subtitle line beneath the logo that contains a unique identifier that includes the student's name and project version.

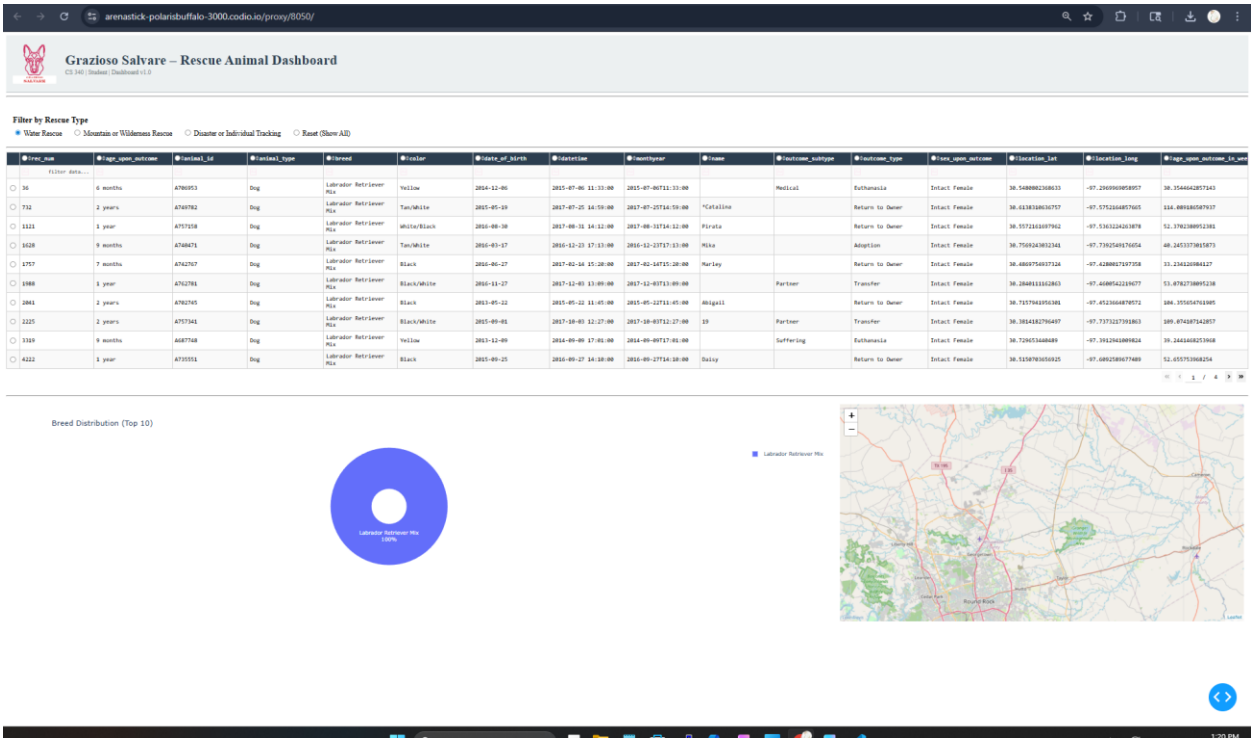
3. Dashboard Screenshots

The following screenshots demonstrate the dashboard functioning correctly. Each image shows the Grazioso Salvare logo and the unique identifier.

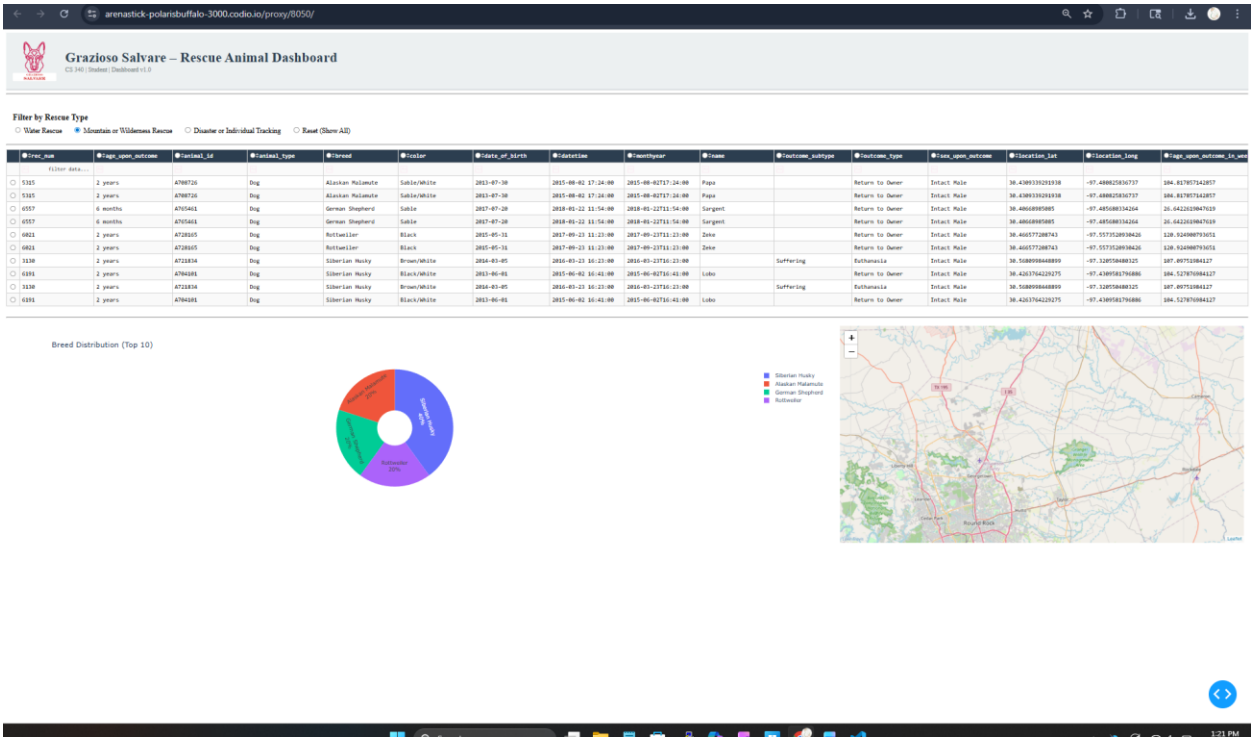
3.1 Starting State (Reset / All Animals)



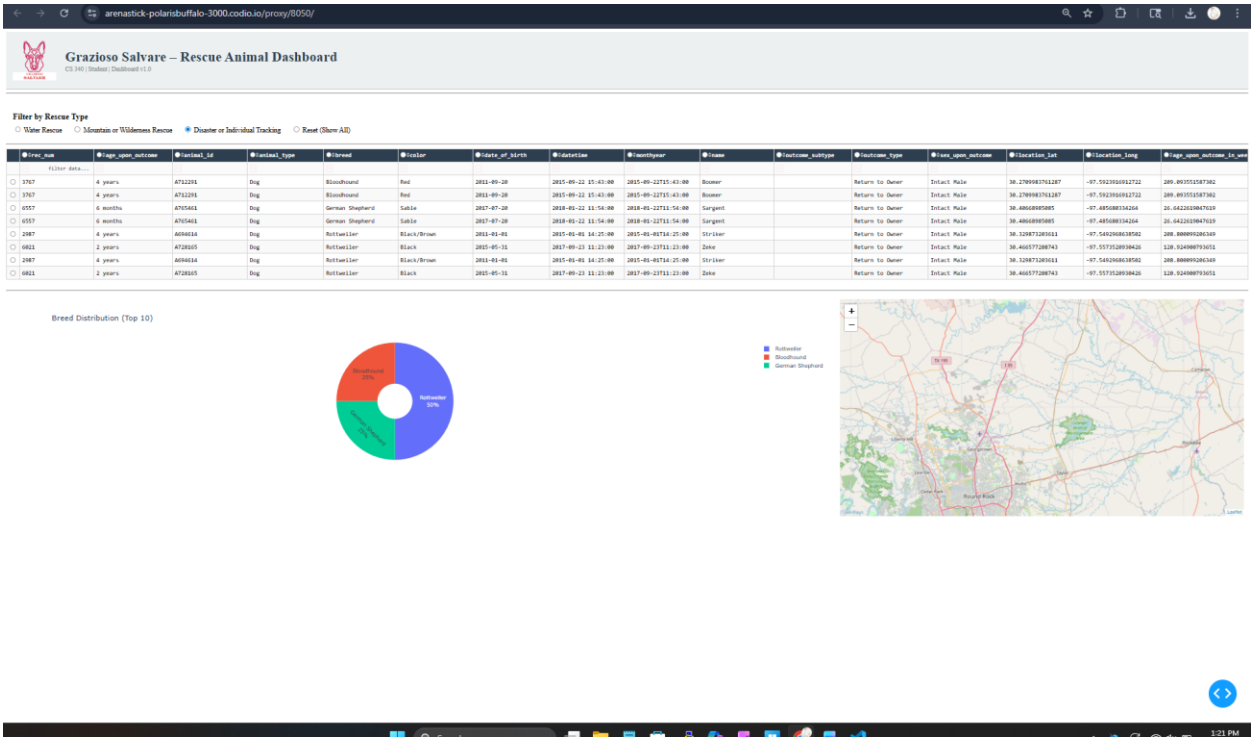
3.2 Water Rescue Filter Applied



3.3 Mountain or Wilderness Rescue Filter Applied



3.4 Disaster or Individual Tracking Filter Applied



4. Tools Used and Rationale

4.1 MongoDB

For a number of reasons, MongoDB was chosen as the project's model component. Because of its document-oriented storage approach, every shelter-animal record is kept as a BSON document, which can be easily mapped to Pandas DataFrames and Python dictionaries without the need for an object-relational mapping layer. The Austin Animal Center dataset, which has optional columns like location_lat, location_long, and age_upon_outcome_in_weeks, is supported by its dynamic schema without necessitating schema migrations when new features are added. Integration with the AnimalShelter module is simple thanks to the official PyMongo driver, which offers a clear, Pythonic API for all CRUD activities. Complex breed-and-age filters can be succinctly written as Python dictionaries instead of raw SQL strings thanks to MongoDB's sophisticated query language, which includes operators like $in, $gte, and $lte.

Lastly, MongoDB collections may scale horizontally through sharding with little modification to the application layer as more shelter partners are added.

4.2 Dash (Plotly)

The MVC architecture's view and controller layers are supplied by Dash. The layout is rendered as a reactive single-page web application by the dash_table and html modules without requiring the developer to be familiar with JavaScript. By connecting user actions like row clicks and radio button selections to database queries and widget updates, dash callbacks—decorated with @app.callback—serve as the controller. The Dash application is rendered inline inside a Jupyter Notebook cell using the JupyterDash extension, which is the necessary delivery format for this project. The breed-distribution pie chart is created using Plotly Express, which produces interactive SVG visuals with little boilerplate. Tile-based interactive maps can be rendered using Dash-Leaflet, an open-source Leaflet.js wrapper for Dash that doesn't require a premium API key.

4.3 Python and Pandas

The main programming language used for the project is Python 3.11. MongoDB cursor results are transformed into DataFrames for simple processing and serialization to the dictionary format needed by the Dash DataTable component using the Pandas package.

4.4 Resources and Links

The following resources were accessed during development:

MongoDB Documentation: https://www.mongodb.com/docs/

PyMongo Documentation: https://pymongo.readthedocs.io/

Plotly Dash Documentation: https://dash.plotly.com/

Dash-Leaflet Documentation: https://www.dash-leaflet.com/

Plotly Express Documentation: https://plotly.com/python/plotly-express/

Austin Animal Center Outcomes Dataset: https://doi.org/10.26000/025.000001

5. Steps to Reproduce the Project

5.1 Prerequisites

Make sure MongoDB is running on localhost port 27017 and that Python 3.8 or later and MongoDB 5.0 or later are installed before beginning the project. Additionally needed are the following Python packages: plotly, pandas, numpy, dash, dash_leaflet, pymongo, and jupyter_dash. One command can be used to install all of these: pip install pymongo jupyter-dash dash dash-leaflet plotly pandas numpy.

5.2 Database Setup

Run the following command from a terminal to import the Austin Animal Center Outcomes CSV file into MongoDB: mongoimport --db aac --collection animals --type csv --headerline --file aac_shelter_outcomes.csv. After the data has been imported, launch the MongoDB shell and use admin then db to create the aacuser account in the admin database with read-write access to the aac database.createUser({ user: "aacuser", pwd: "SNHU1234", roles: [{ role: "readWrite", db: "aac" }] }). The AnimalShelter.py module reflects that this user's authSource is admin.

5.3 Running the Dashboard

Put the Grazioso Salvare Logo PNG file (called "Grazioso Salvare Logo.png"), ProjectTwoDashboard.ipynb, and AnimalShelter.py in the same directory. Launch the notebook

cell by opening ProjectTwoDashboard.ipynb in Jupyter Lab or Jupyter Notebook. The dashboard will show up inline in the browser after the Dash server has started. To alternate between rescue-type filters and the Reset option, use the radio buttons. To update the geolocation map marker to that animal's location, click any row in the data table.

6. Challenges Encountered and Solutions

6.1 MongoDB Authentication Failure

Because the aacuser account had been created across several databases with inconsistent settings and the authSource parameter in the MongoClient function did not match any legitimate user record, the first notebook raised an AuthenticationFailed exception. By removing any duplicate aacuser entries from the AAC, aac, acc, and admin databases and creating a single clean user in the admin database, the problem was fixed. The explicit authMechanism option was then eliminated to enable PyMongo to negotiate the mechanism automatically, and the AnimalShelter constructor was modified to provide authSource="admin" to MongoClient.

6.2 Missing Logo File

The placeholder filename "my-image.png" did not exist in the working directory, therefore the starter code raised a FileNotFoundError. This was fixed by renaming the variable to reference "Grazioso Salvare Logo.png" to match the real file supplied in the Codio environment, changing the code to check os.path.exists() before attempting to open the logo file, and provide a graceful fallback in the event that the file is missing.

6.3 ObjectId Serialization Error

Every document returned by MongoDB has a _id column of type ObjectId, which the Dash DataTable is unable to convert to JSON. This was fixed by giving {'_id': False} as a projection in AnimalShelter's read() method, which excluded the _id field at the PyMongo cursor level.Py. For backward compatibility in the event that the field appears through other code paths, a defensive df.drop() function was also kept in the notebook.

6.4 Map Marker Index Errors

The initial code accessed latitude and longitude information using hard-coded column indices like iloc[row, 13], which resulted in index errors anytime the column order changed as a result of filtering or DataFrame manipulation. To make the code resilient to schema changes, the update_map callback was redesigned to access columns by name (location_lat, location_long, breed, and name) and to incorporate a protective column-existence check before attempting to render the marker.